

Recurrent Neural Networks

Yasser Hifny

Faculty of Computers and Artificial Intelligence
Capital University

Contents

- 1 Model
- 2 Learning Problem
- 3 The Difficulty of Training Simple RNN
- 4 Long Short-Term Memory Networks
 - Vanishing/Exploding Gradients with LSTMs
- 5 Independently Recurrent Neural Network
- 6 Bidirectional Recurrent Neural Networks
- 7 An Example
- 8 Assignments

Recurrent Neural Networks

- Recurrent neural networks (RNNs) process sequential data by maintaining a hidden state.
- The hidden state provides a representation of information accumulated from previous time steps.
- The same parameters are reused at every time step.
- The source introduces simple RNNs, their learning problem, and the difficulty of long-range gradient propagation.

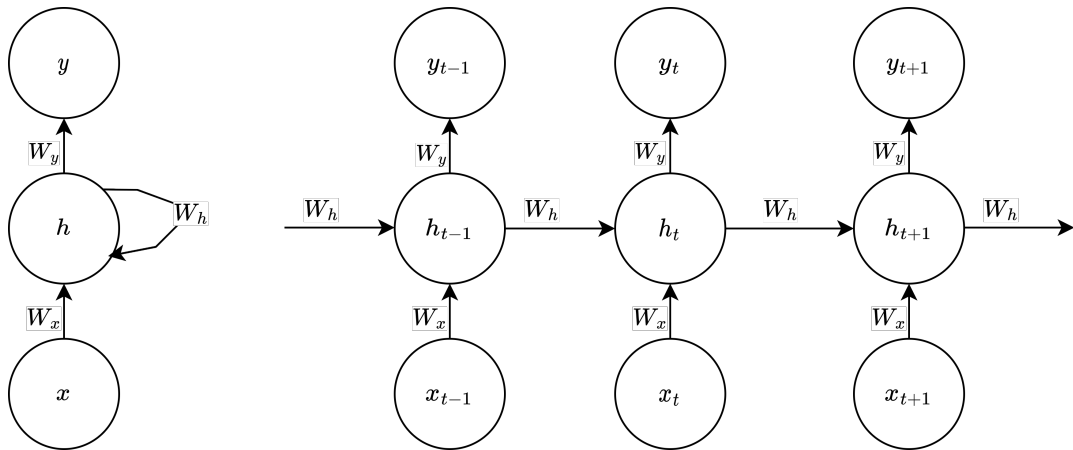
General Recurrent Model

- A recurrent model can be described as a dynamical system:

$$h_t = f_h(X_t, h_{t-1}), \quad y_t = f_o(h_t).$$

- X_t is the input at time t .
- h_t is the hidden state.
- y_t is the output.

Simple RNN Architecture



Simple RNN: Forward Equations

$$z_t^h = W_h h_{t-1} + W_x x_t + b_h,$$

$$z_t^y = W_y h_t + b_y,$$

$$h_t = \tanh(z_t^h)$$

$$y_t = \text{softmax}(z_t^y)$$

- $\text{softmax}(z_i^y) = \frac{e^{z_i^y}}{\sum_j e^{z_j^y}}$

Activation Functions

- The hidden state uses the hyperbolic tangent:

$$h_t = \tanh(z_t^h).$$

- Its derivative is

$$\frac{d}{dx} \tanh(x) = 1 - \tanh^2(x).$$

- The output layer uses softmax to obtain a probability distribution.

Dimensions and Parameters

- x_t : input vector.
- h_{t-1} and h_t : hidden-state vectors.
- W_x : input-to-hidden weights.
- W_h : recurrent hidden-to-hidden weights.
- W_y : hidden-to-output weights.
- b_h, b_y : hidden and output biases.
- At $t = 0$, the source initializes h_{t-1} to zero.

$$z_t^h = W_h h_{t-1} + W_x x_t + b_h, \quad z_t^y = W_y h_t + b_y.$$

Parameter Sharing Across Time

- W_x , W_h , W_y , b_h , and b_y are shared across all time steps.
- Sharing allows the same recurrent mechanism to process sequences of different lengths.
- During training, gradients from all time steps contribute to the same parameters.

Loss Function: Per-Step and Sequence

$$E_t = - \sum_i l_{ti} \log y_{ti}$$

$$E = \sum_{t=1}^T E_t(l_t, y_t) = - \sum_{t=1}^T l_t^T \log y_t = - \sum_{t=1}^T l_t^T \log [\text{softmax}(z_t^y)]$$

Output Error Signal (δ_t^y)

$$\begin{aligned}\delta_t^y &= \frac{\partial E_t}{\partial z_t^y} \\ &= \frac{\partial E_t}{\partial y_t} \times \frac{\partial y_t}{\partial z_t^y} \\ &= y_t - l_t\end{aligned}$$

Hidden-State Error Signal (δ_t^h)

$$\begin{aligned}\delta_t^h &= \frac{\partial E_t}{\partial z_t^h} \\ &= \frac{\partial E_t}{\partial h_t} \frac{\partial h_t}{\partial z_t^h} = \left(\frac{\partial E_t}{\partial z_t^y} \frac{\partial z_t^y}{\partial h_t} + \frac{\partial E_t}{\partial z_{t+1}^h} \frac{\partial z_{t+1}^h}{\partial h_t} \right) \frac{\partial h_t}{\partial z_t^h} \\ &= \left(W_y^T \delta_t^y + W_h^T \delta_{t+1}^h \right) \odot (1 - \tanh^2(z_t^h))\end{aligned}$$

Parameter Gradients

$$\frac{\partial E}{\partial W_y} = \sum_{t=1}^T \delta_t^y \cdot h_t^T,$$

$$\frac{\partial E}{\partial b_y} = \sum_{t=1}^T \delta_t^y$$

$$\frac{\partial E}{\partial W_h} = \sum_{t=1}^T \delta_t^h \cdot h_{t-1}^T,$$

$$\frac{\partial E}{\partial W_x} = \sum_{t=1}^T \delta_t^h \cdot x_t^T$$

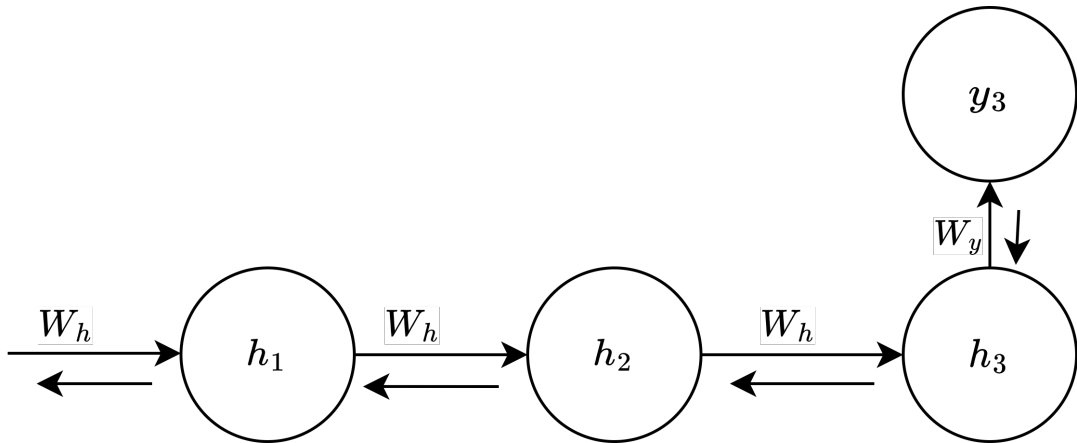
$$\frac{\partial E}{\partial b_h} = \sum_{t=1}^T \delta_t^h$$

Parameter Updates

$$W = W - \eta \frac{\partial E}{\partial W}, \quad b = b - \eta \frac{\partial E}{\partial b}.$$

- η is the learning rate.
- The recurrent parameters are updated after accumulating contributions over the sequence.

Backpropagation Through Time



Three-Step BPTT Example

$$\frac{\partial E_3}{\partial W_h} = \frac{\partial E_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial W_h} + \frac{\partial E_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial W_h} + \frac{\partial E_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial W_h},$$

BPTT Gradient (Single Time Step t)

$$\frac{\partial h_t}{\partial h_{t-1}} = \frac{\partial h_t}{\partial z_t} \cdot \frac{\partial z_t}{\partial h_{t-1}} = f'(z_t) \cdot W_h$$

$$\frac{\partial E_t}{\partial W_h} = \sum_{k=1}^t \frac{\partial E_t}{\partial y_t} \frac{\partial y_t}{\partial h_t} \frac{\partial h_t}{\partial h_k} \frac{\partial h_k}{\partial W_h} = \sum_{k=1}^t \frac{\partial E_t}{\partial y_t} \frac{\partial y_t}{\partial h_t} \left(\prod_{j=k+1}^t \frac{\partial h_j}{\partial h_{j-1}} \right) \frac{\partial h_k}{\partial W_h}$$

where

$$\prod_{j=k+1}^t \frac{\partial h_j}{\partial h_{j-1}} = \prod_{j=k+1}^t W_h^\top \text{diag}[f'(h_j)] = \frac{\partial h_t}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial h_{t-2}} \dots \frac{\partial h_{k+1}}{\partial h_k}.$$

BPTT Gradient (Full Sequence)

$$\begin{aligned}\frac{\partial E}{\partial W_h} &= \sum_{t=1}^T \sum_{k=1}^t \frac{\partial E_t}{\partial y_t} \frac{\partial y_t}{\partial h_t} \frac{\partial h_t}{\partial h_k} \frac{\partial h_k}{\partial W_h} \\ &= \sum_{t=1}^T \left(\sum_{k=1}^t \frac{\partial E_t}{\partial h_k} \prod_{j=k+1}^t W_h^\top \cdot \text{diag}[f'(W_x x_j + W_h h_{j-1} + b_h)] \cdot f'(W_x x_k + W_h h_{k-1} + b_h) h_{k-1}^\top \right)\end{aligned}$$

Bounding the Gradient Norm

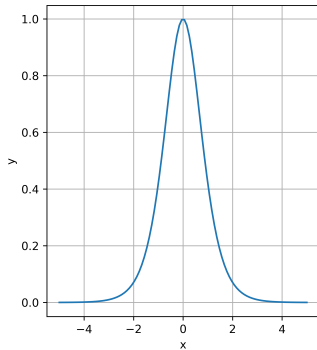
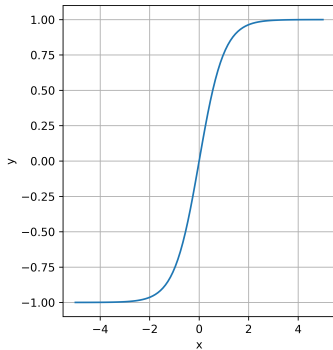
$$\begin{aligned}\left\| \frac{\partial h_j}{\partial h_{j-1}} \right\| &\leq \left\| W_h^\top \right\| \left\| \text{diag}[f'(W_x x_j + W_h h_{j-1} + b_h)] \right\| \leq \gamma_W \gamma_h \\ \left\| \frac{\partial h_t}{\partial h_k} \right\| &= \left\| \prod_{j=k+1}^t \frac{\partial h_j}{\partial h_{j-1}} \right\| \leq (\gamma_W \gamma_h)^{t-k}\end{aligned}$$

Vanishing and Exploding Gradients

$$\left\| \frac{\partial h_t}{\partial h_k} \right\| \leq (\gamma_W \gamma_h)^{t-k}.$$

- If $\gamma_W \gamma_h < 1$, the product can shrink rapidly with the temporal distance.
- If $\gamma_W \gamma_h > 1$, the product can grow rapidly.
- These behaviors lead to vanishing or exploding gradients.
- For the tanh activation, the source uses $\gamma_h = 1$ as an upper bound.
- For the sigmoid activation, the source notes $\gamma_h = 0.25$.
- The recurrent weight norm therefore strongly affects long-range gradient propagation.

Tanh Derivative

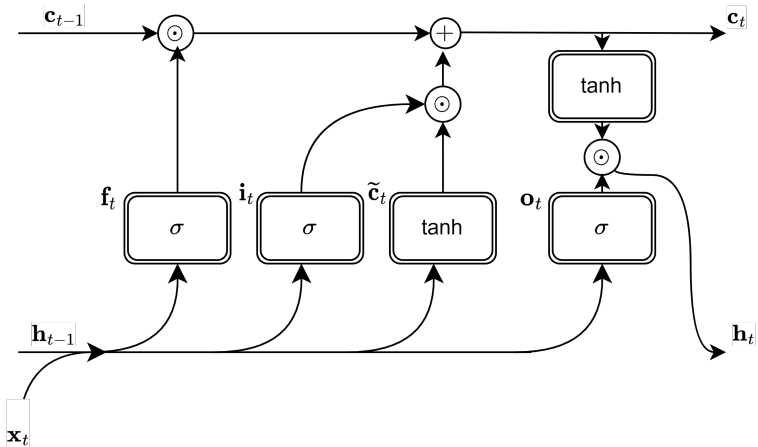


$$f'(x) = 1 - \tanh^2(x).$$

Practical Responses

- Gradient clipping can limit excessively large gradients.
- More structured recurrent architectures were introduced to improve learning over long temporal dependencies.
- The source next discusses LSTMs and related recurrent architectures.

LSTM Architecture



LSTM: Memory and Gates

- LSTM introduces a cell state $\mathbf{c}_t$ that provides an additive path through time.
- Three gates control information flow:
 - input gate $\mathbf{i}_t$,
 - forget gate $\mathbf{f}_t$,
 - output gate $\mathbf{o}_t$.
- A candidate cell content $\tilde{\mathbf{c}}_t$ is combined with the previous cell state.

LSTM Gate Equations

$$\mathbf{i}_t = \sigma(\mathbf{W}_i \mathbf{h}_{t-1} + \mathbf{U}_i \mathbf{x}_t)$$

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \mathbf{h}_{t-1} + \mathbf{U}_f \mathbf{x}_t)$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o \mathbf{h}_{t-1} + \mathbf{U}_o \mathbf{x}_t)$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c \mathbf{h}_{t-1} + \mathbf{U}_c \mathbf{x}_t)$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

LSTM Hidden-State Update

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t).$$

- The cell state carries information forward.
- Gates regulate writing, retaining, and exposing information.

GRU Equations

$$\mathbf{z}_t = \sigma(\mathbf{W}_z \mathbf{h}_{t-1} + \mathbf{U}_z \mathbf{x}_t),$$

$$\mathbf{r}_t = \sigma(\mathbf{W}_r \mathbf{h}_{t-1} + \mathbf{U}_r \mathbf{x}_t),$$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h (\mathbf{h}_{t-1} \odot \mathbf{r}_t) + \mathbf{U}_h \mathbf{x}_t),$$

$$\mathbf{h}_t = \mathbf{z}_t \odot \mathbf{h}_{t-1} + (1 - \mathbf{z}_t) \odot \tilde{\mathbf{h}}_t.$$

LSTM Cell-State Derivative

$$\begin{aligned}\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} &= \frac{\partial \mathbf{f}_t}{\partial \mathbf{c}_{t-1}} \cdot \mathbf{c}_{t-1} + \mathbf{f}_t + \frac{\partial \mathbf{i}_t}{\partial \mathbf{c}_{t-1}} \cdot \tilde{\mathbf{c}}_t + \frac{\partial \tilde{\mathbf{c}}_t}{\partial \mathbf{c}_{t-1}} \cdot \mathbf{i}_t \\ &= \sigma'(\mathbf{W}_f \mathbf{h}_{t-1} + \mathbf{U}_f \mathbf{x}_t) \cdot \mathbf{W}_f \cdot \mathbf{o}_{t-1} \odot \tanh'(\mathbf{c}_{t-1}) \cdot \mathbf{c}_{t-1} \\ &\quad + \mathbf{f}_t \\ &\quad + \sigma'(\mathbf{W}_i \mathbf{h}_{t-1} + \mathbf{U}_i \mathbf{x}_t) \cdot \mathbf{W}_i \cdot \mathbf{o}_{t-1} \odot \tanh'(\mathbf{c}_{t-1}) \cdot \tilde{\mathbf{c}}_t \\ &\quad + \sigma'(\mathbf{W}_c \mathbf{h}_{t-1} + \mathbf{U}_c \mathbf{x}_t) \cdot \mathbf{W}_c \cdot \mathbf{o}_{t-1} \odot \tanh'(\mathbf{c}_{t-1}) \cdot \mathbf{i}_t,\end{aligned}$$

where $\frac{\partial \mathbf{h}_{t-1}}{\partial \mathbf{c}_{t-1}} = \mathbf{o}_{t-1} \odot \tanh'(\mathbf{c}_{t-1})$. Hence, the cell state gradient is an additive function of the four terms computed in the above equation. During the backpropagation, it is possible for these additive terms to have a value of $\vec{1}$. Therefore, using LSTMs, the neural network is trained to determine when the gradient should vanish and when it should be retained by adjusting the values of the four terms.

Gradient Flow Through the Cell State

- The cell-state derivative contains an additive $\mathbf{f}_t$ term.
- The cell state therefore provides a direct path for information and gradient propagation.
- The source nevertheless notes that gradient clipping can still be useful when gradients become large.

$$\frac{\partial \mathbf{h}_{t-1}}{\partial \mathbf{c}_{t-1}} = \mathbf{o}_{t-1} \odot \tanh'(\mathbf{c}_{t-1}).$$

IndRNN: Per-Neuron Recurrence

- Independently Recurrent Neural Networks (IndRNNs) use an independent recurrent coefficient for each neuron.

$$h_{i,t} = f(W_x x_t + u_i h_{i,t-1} + b_i).$$

IndRNN: Gradient Derivation

$$\frac{\partial E}{\partial h_{i,t}} = \sum_{k=t+1}^T \frac{\partial E}{\partial h_{i,k}} \cdot \frac{\partial h_{i,k}}{\partial h_{i,t}}$$

$$\frac{\partial h_{i,k}}{\partial h_{i,t}} = \prod_{j=t+1}^k \frac{\partial h_{i,j}}{\partial h_{i,j-1}}$$

$$\frac{\partial h_{i,j}}{\partial h_{i,j-1}} = u_i f'(W_x x_j + u_i h_{i,j-1} + b_i)$$

ReLU Activation in IndRNN

$$f(x) = \max(0, x), \quad f'(x) = \begin{cases} 1, & x > 0, \\ 0, & \text{otherwise.} \end{cases}$$

- With an independent recurrent coefficient u_i , the temporal gradient can be controlled at the neuron level.

Bidirectional RNN

- A forward recurrent network reads the sequence from left to right.
- A backward recurrent network reads the sequence from right to left.
- Their hidden states can be merged at each time step.

Bidirectional Recurrence and Merging

$$\vec{\mathbf{h}}_t = f(\mathbf{x}_t, \vec{\mathbf{h}}_{t-1})$$

(forward recurrence)

$$\overleftarrow{\mathbf{h}}_t = f(\mathbf{x}_t, \overleftarrow{\mathbf{h}}_{t+1})$$

(backward recurrence)

$$\mathbf{h}_t = \vec{\mathbf{h}}_t + \overleftarrow{\mathbf{h}}_t$$

(merge: summation)

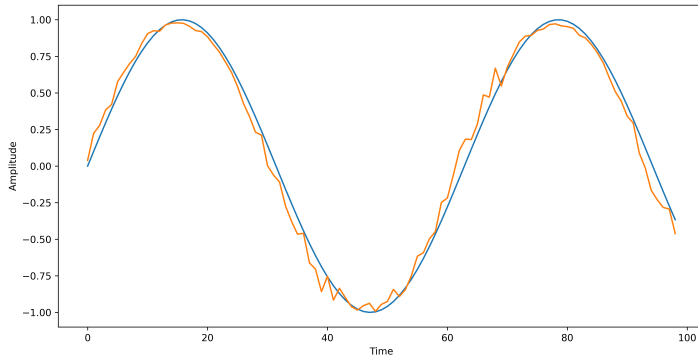
$$\mathbf{h}_t = \begin{bmatrix} \vec{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t \end{bmatrix}$$

(merge: concatenation)

Bidirectional Representation

- Summation keeps the hidden dimensionality unchanged when both directions have the same size.
- Concatenation preserves both directional representations and doubles the hidden dimensionality when the two directions have equal size.
- The appropriate merge depends on the downstream architecture.

Sine-Wave Prediction



Sine wave prediction using the simple RNN (Elman's network).

Sine Example: Model and Data

- Sequence: $y = \sin(0.1x)$.
- Sequence length: 100.
- Training input: all values except the final one.
- Training target: the sequence shifted by one time step.
- Input size: 1; hidden size: 16; output size: 1.
- Learning rate: 0.001; number of epochs: 10,000.

$$h_{t+1} = \tanh(W_{xh}x_t + W_{hh}h_t + b_h), \quad y_t = W_{hy}h_{t+1} + b_y.$$

Python Example: Elman RNN (1/3)

```
import numpy as np
import matplotlib.pyplot as plt

# Define the sine wave sequence
def generate_sequence(length):
    freq = 0.1 # Frequency of the sine wave
    x = np.arange(0, length)
    y = np.sin(freq * x)
    return y

# Define the ElmanRNN class
class ElmanRNN:
    def __init__(self, input_size, hidden_size, output_size):
        self.input_size = input_size
        self.hidden_size = hidden_size
        self.output_size = output_size

        # Initialize the weights
        self.W_xh = np.random.randn(hidden_size, input_size)
        self.W_hh = np.random.randn(hidden_size, hidden_size)
        self.W_hy = np.random.randn(output_size, hidden_size)

        # Initialize the biases
        self.b_h = np.zeros((hidden_size, ))
        self.b_y = np.zeros((output_size, ))

    def forward(self, x):
```

Python Example: Elman RNN (2/3)

```
dL_db_h = np.zeros_like(self.b_h)
dL_db_y = np.zeros_like(self.b_y)
dh_next = np.zeros_like(self.h[0])

for t in reversed(range(T)):
    dL_dy = 2 * (self.y[t] - y[t])
    dL_dW_hy += np.dot(dL_dy.T, self.h[t+1].reshape(-1, 1).T)
    dL_db_y += dL_dy

    dh = np.dot(self.W_hy.T, dL_dy.T) + dh_next.reshape(-1, 1)
    dh_raw = (1 - self.h[t+1] ** 2) * dh[0]
    dL_db_h += dh_raw

    dL_dW_hh += np.dot(dh_raw.reshape(-1, 1), self.h[t].reshape(1, -1))
    dL_dW_xh += np.dot(dh_raw.reshape(-1, 1), x[t].reshape(1, -1))

    dh_next = np.dot(self.W_hh.T, dh_raw.reshape(-1, 1))

# Update weights and biases
self.W_xh -= learning_rate * dL_dW_xh
self.W_hh -= learning_rate * dL_dW_hh
self.W_hy -= learning_rate * dL_dW_hy
self.b_h -= learning_rate * dL_db_h
self.b_y -= learning_rate * dL_db_y
```

Python Example: Elman RNN (3/3)

```
train_data = sequence[:-1]
train_target = sequence[1:]

# Reshape the training data for input to the Elman RNN
train_data = train_data.reshape(sequence_length-1, 1)
train_target = train_target.reshape(sequence_length-1, 1)

# Define hyperparameters
input_size = 1
hidden_size = 16
output_size = 1
learning_rate = 0.001
num_epochs = 10000

# Initialize the Elman RNN model
model = ElmanRNN(input_size, hidden_size, output_size)

# Train the model
model.train(train_data, train_target, learning_rate, num_epochs)

# Generate predictions for the sequence
predictions = model.forward(train_data)

# Plot the original sine wave and the predicted sine wave
plt.figure(figsize=(12, 6))
plt.plot(sequence[:-1], label='Original Sine Wave') # Original sine wave
plt.plot(predictions, label='Predicted Sine Wave') # Predicted sine wave
```

Assignment: RNN Language Modeling

- Implement a neural language-modeling algorithm using an RNN.
- Use the Keras deep-learning library and Google Colab.
- Use the Tiny Shakespeare text dataset.
- Dataset: [github.com/karpathy/char-rnn/.../tinyshakespeare/input.txt](https://github.com/karpathy/char-rnn/blob/master/tinyshakespeare/input.txt)

Key Takeaways

- A simple RNN maintains a recurrent hidden state and shares parameters across time.
- BPTT produces products of Jacobians across time steps.
- Repeated Jacobian products can produce vanishing or exploding gradients.
- LSTM introduces gated additive memory through the cell state.
- IndRNN controls recurrence independently for each neuron.
- Bidirectional RNNs combine forward and backward temporal context.
- The source concludes with a sine-wave prediction example and an RNN language-modeling assignment.

Thank You

Questions?